R.M.K
GROUP OF ENGINEERING INSTITUTIONS



R.M.K
GROUP OF INSTITUTIONS







R.M.K
GROUP OF
INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS

Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

22CS702
DATA ANALYTICS

Department: IT
Batch/Year: 2023-2027 /III YEAR
Created by:
Ms.N.Arockia Rosy/Asst.Professor/IT

Date: 14 - 07 - 2025

R.M.K GROUP OF INSTITUTIONS

1. Table of Contents

S.NO	Topic	Page No.
1.	Contents	5
2.	Course Objectives	6
3.	Pre-Requisites	7
4.	Syllabus	8
5.	Course outcomes	9
6.	CO- PO/PSO Mapping	11
7.	Lecture Plan	13
8.	Activity based learning	14
9.	Lecture notes	16
10.	Assignments	55
11.	Part A Q & A	57
12.	Part B Qs	64
13.	Supportive online Certification courses	65
14.	Real time Applications in day to day life and to Industry	66
15	Contents beyond the Syllabus	68
16	Assessment Schedule	71
17	Prescribed Text Books and Reference Books	72
18	Mini Project Suggestions	73

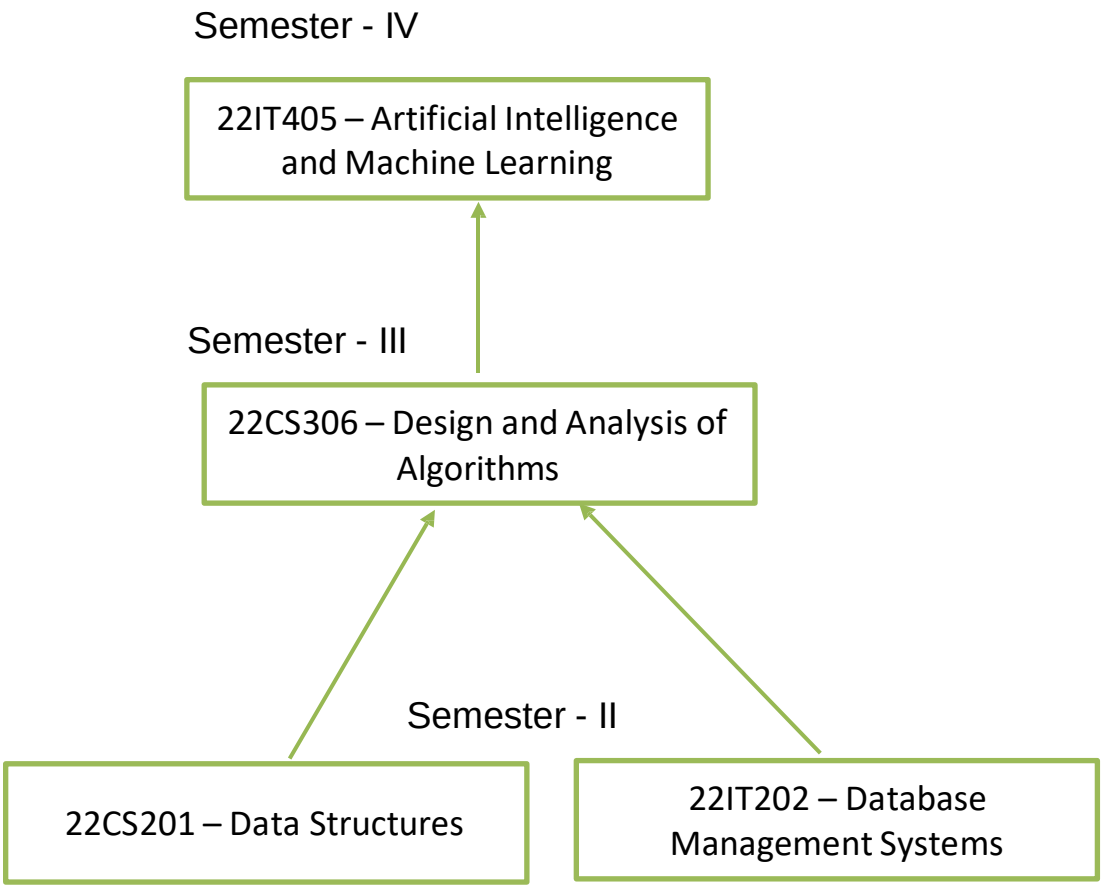
R.M.K GROUP OF INSTITUTIONS

2. Course Objectives

- To explain the fundamentals of big data and data analytics
- To discuss the Hadoop framework
- To explain about exploratory data analysis and data manipulation tools
- To analyse and interpret streaming data
- To discuss various applications of data analytics

R.M.K GROUP OF INSTITUTIONS

3. Pre-Requisites



4. SYLLABUS

22CS702 **DATA ANALYTICS** **L T P C**

3 0 0 3

UNIT I INTRODUCTION 12

Evolution of Big Data-Definition of Big Data-Challenges with Big Data- Traditional Business Intelligence (BI) versus Big Data-Introduction to big data analytics-Classification of Analytics-Analytics Tools- Importance of big data analytics.

List of Exercise/Experiments:

1. Download, install and explore the features of R/Python for data analytics.
2. Working with Numpy arrays

UNIT II HADOOP FRAMEWORK 12

Introducing Hadoop- RDBMS versus Hadoop- Hadoop Overview-HDFS (Hadoop Distributed File System)- Processing Data with Hadoop- Managing Resources and Applications with Hadoop YARN - Interacting with Hadoop Ecosystem.

List of Exercise/Experiments:

1. Working with Pandas data frames
2. Basic plots using Matplotlib

UNIT III EXPLORATORY DATA ANALYSIS 12

EDA fundamentals – Understanding data science – Significance of EDA – Making sense of data – Comparing EDA with classical and Bayesian analysis – Software tools for EDA –Data transformation techniques - Introduction to NoSQL – MongoDB: RDBMS Vs MongoDB – Data Types – Query Language – Hive – Hive Architecture – Data Types – File Formats - Hive Query Language (HQL) – RC File Implementation – User Defined Functions.

List of Exercise/Experiments:

1. Statistical and Probability measures - Frequency distributions, Mean, Mode, Standard Deviation, Variability, Normal curves, Correlation and scatter plots, Correlation coefficient, Regression.
2. Use the standard benchmark data set for performing the following:

a) Univariate Analysis: Frequency, Mean, Median, Mode, Variance, Standard Deviation, Skewness and Kurtosis.

b) Bivariate Analysis: Linear and logistic regression modelling.

c) Multiple Regression Analysis

d) Compare the results of the above analysis for the two data sets

TOTAL: 45 PERIODS

R.M.K GROUP OF INSTITUTIONS

UNIT IV MINING DATA STREAMS 12

The data stream model – stream queries-sampling data in a stream-general streaming problem- filtering streams-analysis of filtering- dealing with infinite streams- Counting Distance Elements in a Stream – Estimating Moments – Counting Ones in Window – Decaying Windows.

List of Exercise/Experiments:

1. Apply and explore various plotting functions on any data set.
2. Implement the following algorithms on real time stream data sets.
   Support Vector Machine

Decision tree classifier

Clustering Algorithms

UNIT V APPLICATIONS 12

Application: Sales and Marketing – Industry Specific Data Mining – microRNA Data Analysis Case Study – Credit Scoring Case Study – Data Mining Nontabular Data.

List of Exercise/Experiments:

1. Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using the standard Heart Disease Data Set. You can use Java/Python ML library classes/API
2. Mini Project: The project should contain the following components

Realtime dataset

Data preparation & Transformation

Handling missing Data

Data Storage

Algorithm for data analytics

Data visualization: Charts, Heatmap, Crosstab, Treemap

R.M.K GROUP OF INSTITUTIONS

5. COURSE OUTCOMES

At the end of this course, the students will be able to:

COURSE OUTCOMES		HKL
CO1	Explain the fundamentals of big data and data analytics and illustrate it.	K2
CO2	Discuss the Hadoop framework.	K2
CO3	Develop applications using exploratory data analysis and data manipulation tools.	K3
CO4	Analyze and interpret streaming data.	K4
CO5	Illustrate various applications of data analytics.	K2
CO6	Build applications for various domain.	K6

R.M.K GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

CO – PO/PSO Mapping

R.M.K
GROUP OF
INSTITUTIONS

6.CO – PO /PSO Mapping Matrix

CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PS O1	PS O2	PS 03
1	3	2	1	2	2	-	-	1	2	1	-	3	3	3	3
2	3	2	1	2	3	-	-	1	2	2	-	2	3	3	3
3	3	2	2	3	3	-	-	1	2	1	2	3	3	3	3
4	3	3	2	3	3	-	-	1	2	1	-	3	3	3	3
5	3	3	2	3	1	-	-	1	2	2	2	3	3	3	3
6	3	3	3	3	2	-	-	1	2	2	2	3	3	3	3

R.M.K GROUP OF INSTITUTIONS

Lecture Plan

Unit - IV

R.M.K GROUP OF INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS

LECTURE PLAN – Unit 4 - MINING DATA STREAMS

Sl. No.	Topic	Number of Periods	Proposed Date	Actual Lecture Date	CO	Taxonomy Level	Mode of Delivery
1	The data stream model, stream queries	1			4	K4	
2	sampling data in a stream	1			4	K4	
3	general streaming problem	1			4	K4	
4	filtering streams	1			4	K4	
5	analysis of filtering	1			4	K4	
6	dealing with infinite streams	1			4	K4	
7	Counting Distance Elements in a Stream	1			4	K4	
8	Estimating Moments	1			4	K4	
9	Counting Ones in Window, Decaying Windows	1			4	K4	

8. ACTIVITY BASED LEARNING

Social Media Analysis (Twitter API / X API)

- Stream tweets with specific hashtags (#AI, #ClimateChange, #Sports).
- Visualize tweet frequency, sentiment analysis, or most used keywords in real time.

Guidelines to do an activity :

1) Students can form group. (3 students / team)
2) Set Up Environment: Connect to a real-time data source like Twitter API or a financial market feed.
3) Get Twitter/X API Access , Generate API keys and tokens, then share securely with group members.
4) Collect Tweets
5) Analyze Tweets
6) Create at least two charts and prepare a short group presentation/report.

R.M.K GROUP OF INSTITUTIONS

UNIT-IV

MINING DATA STREAMS

9. LECTURE NOTES

1. The Stream Data Model

1.1 A Data-Stream-Management System

- In analogy to a database-management system, we can view a stream processor as a kind of data-management system, the high-level organization of which is suggested in Fig. 4.1

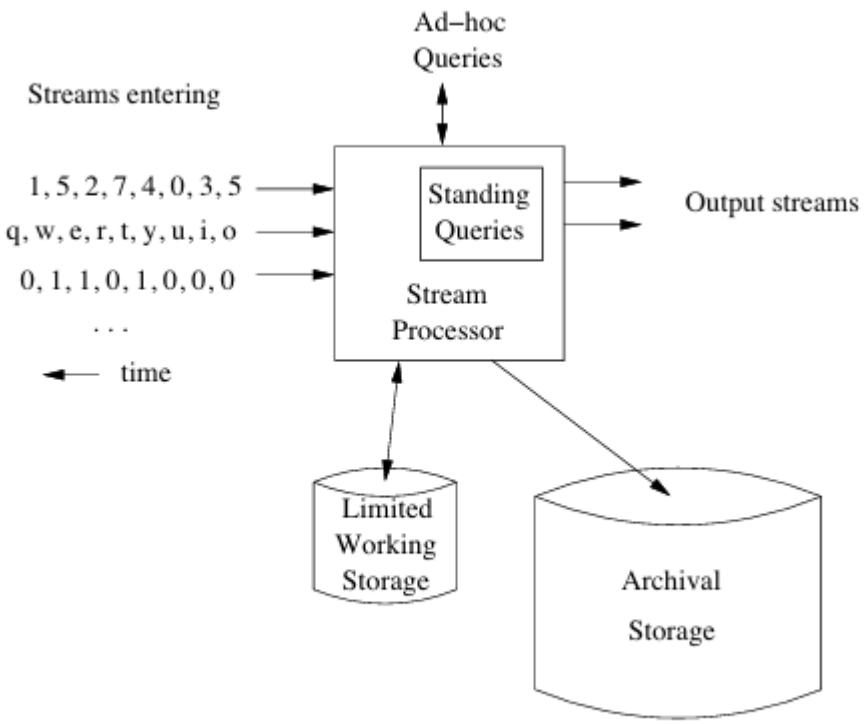


Figure 4.1 A data-stream-management system

- Any number of streams can enter the system.
- Each stream can provide elements at its own schedule; they need not have the same data rates or data types, and the time between elements of one stream need not be uniform.
- The fact that the rate of arrival of stream elements is not under the control of the system distinguishes stream processing from the processing of data that goes on within a database-management system.

R.M.K GROUP OF INSTITUTIONS

- The latter system controls the rate at which data is read from the disk, and therefore no data is lost as it attempts to execute queries.
- Streams may be archived in a large archival store, but we assume it is not possible to answer queries from the archival store. It could be examined only under special circumstances using time-consuming retrieval processes.
- There is also a working store, into which summaries or parts of streams may be placed, and which can be used for answering queries.
- The working store might be disk, or it might be main memory, depending on how fast we need to process queries. But either way, it is of sufficiently limited capacity that it cannot store all the data from all the streams.

1.2 Examples of Stream Sources

Let us consider some of the ways in which stream data arises naturally.

(a) Sensor Data

- Imagine a temperature sensor bobbing about in the ocean, sending back to a base station a reading of the surface temperature each
- The data produced by this sensor is a stream of real numbers. It is not a very interesting stream, since the data rate is so low and the entire stream could be kept in main memory, essentially forever.
- Now, give the sensor a GPS unit, and let it report surface height instead of temperature.
- The surface height varies quite rapidly compared with temperature, so we might have the sensor send back a reading every tenth of a second.
- If it sends a 4-byte real number each time, then it produces 3.5 megabytes per day. It will still take some time to fill up main memory, let alone a single disk.

R.M.K GROUP OF INSTITUTIONS

- But one sensor might not be that interesting. To learn something about ocean behavior, we might want to deploy a million sensors, each sending back a stream, at the rate of ten per second.
- A million sensors isn't very many; there would be one for every 150 square miles of ocean. Now we have 3.5 terabytes arriving every day, and definitely need to think about what can be kept in working storage and what can only be archived.

(b) Image Data

- Satellites often send down to earth streams consisting of many terabytes of images per day.
- Surveillance cameras produce images with lower resolution than satellites, but there can be many of them, each producing a stream of images at intervals like one second. London is said to have six million such cameras, each producing a stream.

(c) Internet and Web Traffic

- A switching node in the middle of the Internet receives streams of IP packets from many inputs and routes them to its outputs.
- Normally, the job of the switch is to transmit data and not to retain it or query it. But there is a tendency to put more capability into the switch, e.g., the ability to detect denial-of-service attacks or the ability to reroute packets based on information about congestion in the network.
- Web sites receive streams of various types. For example, Google receives several hundred million search queries per day. Yahoo! accepts billions of "clicks" per day on its various sites.

R.M.K GROUP OF INSTITUTIONS

- Many interesting things can be learned from these streams. For example, an increase in queries like "sore throat" enables us to track the spread of viruses.
- A sudden increase in the click rate for a link could indicate some news connected to that page, or it could mean that the link is broken and needs to be repaired.

3. Stream Queries

There are two ways that queries get asked about streams. We have seen in Fig. 4.1 a place within the processor where standing queries are stored. These queries are, in a sense, permanently executing, and produce outputs at appropriate times.

Example 4.1: The stream produced by the ocean-surface-temperature sensor mentioned at the beginning might have a standing query to output an alert whenever the temperature exceeds 25 degrees centigrade. This query is easily answered, since it depends only on the most recent stream element.

- Alternatively, we might have a standing query that, each time a new reading arrives, produces the average of the 24 most recent readings.
- That query also can be answered easily, if we store the 24 most recent stream elements. When a new stream element arrives, we can drop from the working store the 25th most recent element, since it will never again be needed (unless there is some other standing query that requires it).
- Another query we might ask is the maximum temperature ever recorded by that sensor. We can answer this query by retaining a simple summary: the maximum of all stream elements ever seen. It is not necessary to record the entire stream.

R.M.K GROUP OF INSTITUTIONS

- When a new stream element arrives, we compare it with the stored maximum, and set the maximum to whichever is larger.
- We can then answer the query by producing the current value of the maximum.
- Similarly, if we want the average temperature over all time, we have only to record two values: the number of readings ever sent in the stream and the sum of those readings.
- We can adjust these values easily each time a new reading arrives, and we can produce their quotient as the answer to the query.

The other form of query is ad-hoc, a question asked once about the current state of a stream or streams.

- If we do not store all streams in their entirety, as normally we can not, then we cannot expect to answer arbitrary queries about
- If we have some idea what kind of queries will be asked through the ad-hoc query interface, then we can prepare for them by storing appropriate parts or summaries of streams as in Example
- If we want the facility to ask a wide variety of ad-hoc queries, a common approach is to store a sliding window of each stream in the working store.
- A sliding window can be the most recent n elements of a stream, for some n, or it can be all the elements that arrived within the last t time units, e.g., one day.
- If we regard each stream element as a tuple, we can treat the window as a relation and query it with any SQL query.
- The stream-management system must keep the window fresh, deleting the oldest elements as new ones come in.

Example 4.2: Web sites often like to report the number of unique users over the past month. If we think of each login as a stream element, we can maintain a window that is all logins in the most recent month. We must associate the arrival time with each login, so we know when it no longer belongs to the window. If we think of the window as a relation Logins(name, time), then it is simple to get the number of unique users over the past month. The SQL query is:

```
SELECT COUNT(DISTINCT(name))
FROM Logins
WHERE time >= t;
```

Here, t is a constant that represents the time one month before the current time. Note that we must be able to maintain the entire stream of logins for the past month in working storage. However, for even the largest sites, that data is not more than a few terabytes, and so surely can be stored on disk.

2. Sampling Data in a Stream

2.1 A Motivating Example

- The general problem we shall address is selecting a subset of a stream so that we can ask queries about the selected subset and have the answers be statistically representative of the stream as a whole.
- If we know what queries are to be asked, then there are a number of methods that might work, but we are looking for a technique that will allow ad-hoc queries on the sample. We shall look at a particular problem, from which the general idea will emerge.

R.M.K GROUP OF INSTITUTIONS

Our running example is the following. A search engine receives a stream of queries, and it would like to study the behavior of typical users. We assume the stream consists of tuples (user, query, time).

- Suppose that we want to answer queries such as "What fraction of the typical user's queries were repeated over the past month?" Assume also that we wish to store only 1/10th of the stream elements.

The obvious approach would be to generate a random number, say an integer from 0 to 9, in response to each search query.

- Store the tuple if and only if the random number is 0. If we do so, each user has, on average, 1/10th of their queries stored.
- Statistical fluctuations will introduce some noise into the data, but if users issue many queries, the law of large numbers will assure us that most users will have a fraction quite close to 1/10th of their queries stored.

However, this scheme gives us the wrong answer to the query asking for the average number of duplicate queries for a user.

- Suppose a user has issued s search queries one time in the past month, d search queries twice, and no search queries more than twice. If we have a 1/10th sample, of queries, we shall see in the sample for that user an expected s/10 of the search queries issued once.
- Of the d search queries issued twice, only d/100 will appear twice in the sample; that fraction is d times the probability that both occurrences of the query will be in the 1/10th sample.
- Of the queries that appear twice in the full stream, 18d/100 will appear exactly once. To see why, note that 18/100 is the probability that one of the two occurrences will be in the 1/10th of the stream that is selected, while the other is in the 9/10th that is not selected.

R.M.K GROUP OF INSTITUTIONS

- The correct answer to the query about the fraction of repeated searches is d/(s+d). However, the answer we shall obtain from the sample is d/(10s+19d).
- To derive the latter formula, note that d/100 appear twice, while s/10+18d/100 appear once. Thus, the fraction appearing twice in the sample is d/100 divided by d/100+s/10+18d/100. This ratio is d/(10s+ 19d). For no positive values of s and d is d/(s + d) = d/(10s + 19d).

7) **Obtaining a Representative Sample**

- The query of Section 2.1, like many queries about the statistics of typical users, cannot be answered by taking a sample of each user's search queries.
- Thus, we must strive to pick 1/10th of the users, and take all their searches for the sample, while taking none of the searches from other users.
- If we can store a list of all users, and whether or not they are in the sample, then we could do the following.
- Each time a search query arrives in the stream, we look up the user to see whether or not they are in the sample. If so, we add this search query to the sample, and if not, then not.
- However, if we have no record of ever having seen this user before, then we generate a random integer between 0 and 9.
- If the number is 0, we add this user to our list with value "in," and if the number is other than 0, we add the user with the value "out."

That method works as long as we can afford to keep the list of all users and their in/out decision in main memory, because there isn't time to go to disk for every search that arrives. By using a hash function, one can avoid keeping the list of users.

R.M.K GROUP OF INSTITUTIONS

That is, we hash each user name to one of ten buckets, 0 through 9. If the user hashes to bucket 0, then accept this search query for the sample, and if not, then not.

Note we do not actually store the user in the bucket; in fact, there is no data in the buckets at all.

- Effectively, we use the hash function as a random-number generator, with the important property that, when applied to the same user several times, we always get the same "random" number.
- That is, without storing the in/out decision for any user, we can reconstruct that decision any time a search query by that user arrives.
- More generally, we can obtain a sample consisting of any rational fraction a/b of the users by hashing user names to b buckets, 0 through b – 1. Add the search query to the sample if the hash value is less than a.

2.3 The General Sampling Problem

The running example is typical of the following general problem.

- Our stream consists of tuples with n
- A subset of the components are the key components, on which the selection of the sample will be based.
- In our running example, there are three components – user, query, and time – of which only user is in the key.
- However, we could also take a sample of queries by making query be the key, or even take a sample of user-query pairs by making both those components form the key.
- To take a sample of size a/b, we hash the key value for each tuple to b buckets, and accept the tuple for the sample if the hash value is less than a.
- If the key consists of more than one component, the hash function needs to combine the values for those components to make a single hash-value.

R.M.K GROUP OF INSTITUTIONS

- The result will be a sample consisting of all tuples with certain key values. The selected key values will be approximately a/b of all the key values appearing in the stream.

8) Filtering Streams

- Another common process on streams is selection, or filtering.
- We want to accept those tuples in the stream that meet a criterion. Accepted tuples are passed to another process as a stream, while other tuples are dropped.
- If the selection criterion is a property of the tuple that can be calculated (e.g., the first component is less than 10), then the selection is easy to do.
- The problem becomes harder when the criterion involves lookup for membership in a set.
- It is especially hard, when that set is too large to store in main memory.
.
- In this section, we shall discuss the technique known as "Bloom filtering" as a way to eliminate most of the tuples that do not meet the criterion.

1. A Motivating Example

- Suppose we have a set S of one billion allowed email addresses – those that we will allow through because we believe them not to be spam.
- The stream consists of pairs: an email address and the email itself. Since the typical email address is 20 bytes or more, it is not reasonable to store S in main memory.
- Thus, we can either use disk accesses to determine whether or not to let through any given stream element, or we can devise a method that requires no more main memory than we have available, and yet will filter most of the undesired stream elements.

Suppose we have one gigabyte of available main memory.

- In the technique known as Bloom filtering, we use that main memory as a bit array.
- In this case, we have room for eight billion bits, since one byte equals eight bits. Devise a hash function h from email addresses to eight billion buckets. Hash each member of S to a bit, and set that bit to 1. All other bits of the array remain 0.
- Since there are one billion members of S, approximately 1/8th of the bits will be 1. The exact fraction of bits set to 1 will be slightly less than 1/8th, because it is possible that two members of S hash to the same bit.
- When a stream element arrives, we hash its email address. If the bit to which that email address hashes is 1, then we let the email through. But if the email address hashes to a 0, we are certain that the address is not in S, so we can drop this stream element.

Unfortunately, some spam email will get through.

- Approximately 1/8th of the stream elements whose email address is not in S will happen to hash to a bit whose value is 1 and will be let through.
- Nevertheless, since the majority of emails are spam (about 80% according to some reports), eliminating 7/8th of the spam is a significant benefit.
- Moreover, if we want to eliminate every spam, we need only check for membership in S those good and bad emails that get through the filter.
- Those checks will require the use of secondary memory to access S itself. As a simple example, we could use a cascade of filters, each of which would eliminate 7/8th of the remaining spam.

R.M.K GROUP OF INSTITUTIONS

2. The Bloom Filter

A Bloom filter consists of:

1. An array of n bits, initially all 0's.
2. A collection of hash functions h1,h2,...,hk.
   Each hash function maps "key" values to n buckets, corresponding to the n bits of the bit-array.
3. A set S of m key values.

- The purpose of the Bloom filter is to allow through all stream elements whose keys are in S, while rejecting most of the stream elements whose keys are not in S.
- To initialize the bit array, begin with all bits 0. Take each key value in S and hash it using each of the k hash functions.
- Set to 1 each bit that is hi(K) for some hash function hi and some key value K in S.
- To test a key K that arrives in the stream, check that all of h1(K),h2(K),...,hk(K

are 1's in the bit-array. If all are 1's, then let the stream element through. If one or more of these bits are 0, then K could not be in S, so reject the stream element.

3.3 Analysis of Bloom Filtering

- If a key value is in S, then the element will surely pass through the Bloom filter.
- However, if the key value is not in S, it might still pass. We need to understand how to calculate the probability of a false positive, as a function of n, the bit-array length, m the number of members of S, and k, the number of hash functions.
- The model to use is throwing darts at targets. Suppose we have x targets and y darts. Any dart is equally likely to hit any target. After throwing the darts, how many targets can we expect to be hit at least once?

R.M.K GROUP OF INSTITUTIONS

The analysis is as follows:

- The probability that a given dart will not hit a given target is $(x-1)/x$.
- The probability that none of the y darts will hit a given target is $\left(\frac{x-1}{x}\right)^y$. We can write this expression as $(1-\frac{1}{x})^{x(\frac{y}{x})}$.
- Using the approximation $(1-\epsilon)^{1/\epsilon} = 1/e$ for small ϵ (recall Section 1.3.5), we conclude that the probability that none of the y darts hit a given target is $e^{-y/x}$.

Example 4.3: Consider the running example of Section 4.3.1. We can use the above calculation to get the true expected number of 1's in the bit array. Think of each bit as a target, and each member of S as a dart. Then the probability that a given bit will be 1 is the probability that the corresponding target will be hit by one or more darts. Since there are one billion members of S, we have $y = 10^9$ darts. As there are eight billion bits, there are $x = 8 \times 10^9$ targets. Thus, the probability that a given target is not hit is $e^{-y/x} = e^{-1/8}$ and the probability that it *is* hit is $1 - e^{-1/8}$. That quantity is about 0.1175. In Section 4.3.1 we suggested that $1/8 = 0.125$ is a good approximation, which it is, but now we have the exact calculation. □

We can apply the rule to the more general situation, where set S has m members, the array has n bits, and there are k hash functions. The number of targets is $x = n$, and the number of darts is $y = km$. Thus, the probability that a bit remains 0 is $e^{-km/n}$. We want the fraction of 0 bits to be fairly large, or else the probability that a nonmember of S will hash at least once to a 0 becomes too small, and there are too many false positives. For example, we might choose k, the number of hash functions to be n/m or less. Then the probability of a 0 is at least e^{-1} or 37%. In general, the probability of a false positive is the probability of a 1 bit, which is $1 - e^{-km/n}$, raised to the kth power, i.e., $(1 - e^{-km/n})^k$.

R.M.K GROUP OF INSTITUTIONS

Example 4.4: In Example 4.3 we found that the fraction of 1's in the array of our running example is 0.1175, and this fraction is also the probability of a false positive. That is, a nonmember of S will pass through the filter if it hashes to a 1, and the probability of it doing so is 0.1175.

Suppose we used the same S and the same array, but used two different hash functions. This situation corresponds to throwing two billion darts at eight billion targets, and the probability that a bit remains 0 is $e^{-1/4}$. In order to be a false positive, a nonmember of S must hash twice to bits that are 1, and this probability is $(1 - e^{-1/4})^2$, or approximately 0.0493. Thus, adding a second hash function for our running example is an improvement, reducing the false-positive rate from 0.1175 to 0.0493. □

9) **Counting Distinct Elements in a Stream**

In this section we look at a third simple kind of processing we might want to do on a stream. As with the previous examples – sampling and filtering – it is somewhat tricky to do what we want in a reasonable amount of main memory, so we use a variety of hashing and a randomized algorithm to get approximately what we want with little space needed per stream.

1. The Count-Distinct Problem

Suppose stream elements are chosen from some universal set. We would like to know how many different elements have appeared in the stream, counting either from the beginning of the stream or from some known time in the past.

Example 4.5: As a useful example of this problem, consider a Web site gathering statistics on how many unique users it has seen in each given month. The universal set is the set of logins for that site, and a stream element is generated each time someone logs in. This measure is appropriate for a site like

R.M.K GROUP OF INSTITUTIONS

Amazon, where the typical user logs in with their unique login name.

- A similar problem is a Web site like Google that does not require login to issue a search query, and may be able to identify users only by the IP address from which they send the query.
- There are about 4 billion IP addresses, 2 sequences of four 8-bit bytes will serve as the universal set in this case.

The obvious way to solve the problem is to keep in main memory a list of all the elements seen so far in the stream.

- Keep them in an efficient search structure such as a hash table or search tree, so one can quickly add new elements and check whether or not the element that just arrived on the stream was already seen.
- As long as the number of distinct elements is not too great, this structure can fit in main memory and there is little problem obtaining an exact answer to the question how many distinct elements appear in the stream.

However, if the number of distinct elements is too great, or if there are too many streams that need to be processed at once (e.g., Yahoo! wants to count the number of unique users viewing each of its pages in a month), then we cannot store the needed data in main
memory.

- There are several options. We could use more machines, each machine handling only one or several of the streams.
- We could store most of the data structure in secondary memory and batch stream elements so whenever we brought a disk block to main memory there would be many tests and updates to be performed on the data in that block.
- Or we could use the strategy where we only estimate the number of distinct elements but use much less memory than the number of distinct elements.

2. The Flajolet-Martin Algorithm

- It is possible to estimate the number of distinct elements by hashing the elements of the universal set to a bit-string that is sufficiently long.
- The length of the bit-string must be sufficient that there are more possible results of the hash function than there are elements of the universal set.
- For example, 64 bits is sufficient to hash URL's. We shall pick many different hash functions and hash each element of the stream using these hash functions.
- The important property of a hash function is that when applied to the same element, it always produces the same result. Notice that this property was also essential for the sampling technique.
- The idea behind the Flajolet-Martin Algorithm is that the more different elements we see in the stream, the more different hash-values we shall see.
- As we see more different hash-values, it becomes more likely that one of these values will be
- The particular unusual property we shall exploit is that the value ends in many 0's, although many other options exist.

Whenever we apply a hash function h to a stream element a, the bit string $h(a)$ will end in some number of 0's, possibly none. Call this number the *tail length* for a and h. Let R be the maximum tail length of any a seen so far in the stream. Then we shall use estimate 2^R for the number of distinct elements seen in the stream.

R.M.K GROUP OF INSTITUTIONS

This estimate makes intuitive sense. The probability that a given stream element a has $h(a)$ ending in at least r 0's is 2^{-r}. Suppose there are m distinct elements in the stream. Then the probability that none of them has tail length at least r is $(1-2^{-r})^m$. This sort of expression should be familiar by now. We can rewrite it as $\left((1-2^{-r})^{2^r}\right)^{m2^{-r}}$. Assuming r is reasonably large, the inner expression is of the form $(1-\epsilon)^{1/\epsilon}$, which is approximately $1/e$. Thus, the probability of not finding a stream element with as many as r 0's at the end of its hash value is $e^{-m2^{-r}}$. We can conclude:

1. If m is much larger than 2^r, then the probability that we shall find a tail of length at least r approaches 1.

2. If m is much less than 2^r, then the probability of finding a tail length at least r approaches 0.

We conclude from these two points that the proposed estimate of m, which is 2^R (recall R is the largest tail length for any stream element) is unlikely to be either much too high or much too low.

4.3 Combining Estimates

Unfortunately, there is a trap regarding the strategy for combining the estimates of m, the number of distinct elements, that we obtain by using many di fferent hash functions.

- Our first assumption would be that if we take the average of the values 2R that we get from each hash function, we shall get a value that approaches the true m, the more hash functions we use.
- However, that is not the case, and the reason has to do with the influence an overestimate has on the average.

Consider a value of r such that 2^r is much larger than m. There is some probability p that we shall discover r to be the largest number of 0's at the end of the hash value for any of the m stream elements. Then the probability of finding $r+1$ to be the largest number of 0's instead is at least $p/2$. However, if we do increase by 1 the number of 0's at the end of a hash value, the value of 2^R doubles. Consequently, the contribution from each possible large R to the expected value of 2^R grows as R grows, and the expected value of 2^R is actually infinite.[3]

Another way to combine estimates is to take the median of all estimates.

- The median is not affected by the occasional outsized value of 2R, so the worry described above for the average should not carry over to the median.
- Unfortunately, the median suffers from another defect: it is always a power of 2. Thus, no matter how many hash functions we use, should the correct value of m be between two powers of 2, say 400, then it will be impossible to obtain a close estimate.

There is a solution to the problem, however. We can combine the two methods.

- First, group the hash functions into small groups, and take their average. Then, take the median of the averages.
- It is true that an occasional outsized 2R will bias some of the groups and make them too large. However, taking the median of group averages will reduce the influence of this effect almost to nothing.
- Moreover, if the groups themselves are large enough, then the averages can be essentially any number, which enables us to approach the true value m as long as we use enough hash functions.
- In order to guarantee that any possible average can be obtained, groups should be of size at least a small multiple of log2 m.

R.M.K GROUP OF INSTITUTIONS

4. Space Requirements

As we read the stream it is not necessary to store the elements seen.

- The only thing we need to keep in main memory is one integer per hash function; this integer records the largest tail length seen so far for that hash function and any stream element.
- If we are processing only one stream, we could use millions of hash functions, which is far more than we need to get a close estimate.
- Only if we are trying to process many streams at the same time would main memory constrain the number of hash functions we could associate with any one stream.
- In practice, the time it takes to compute hash values for each stream element would be the more significant limitation on the number of hash functions we use.

10) Estimating Moments

Here we consider a generalization of the problem of counting distinct elements in a stream.

- The problem, called computing "moments," involves the distribution of frequencies of different elements in the stream.
- We shall define moments of all orders and concentrate on computing second moments, from which the general algorithm for all moments is a simple extension.

1. Definition of Moments

Suppose a stream consists of elements chosen from a universal set. Assume the universal set is ordered so we can speak of the ith element for any i. Let m_i be the number of occurrences of the ith element for any i. Then the *kth-order moment* (or just kth moment) of the stream is the sum over all i of $(m_i)^k$.

R.M.K GROUP OF INSTITUTIONS

Example 4.6: The 0th moment is the sum of 1 for each m_i that is greater than 0.[4] That is, the 0th moment is a count of the number of distinct elements in the stream. We can use the method of Section 4.4 to estimate the 0th moment of a stream.

The 1st moment is the sum of the m_i's, which must be the length of the stream. Thus, first moments are especially easy to compute; just count the length of the stream seen so far.

The second moment is the sum of the squares of the m_i's. It is sometimes called the *surprise number*, since it measures how uneven the distribution of elements in the stream is. To see the distinction, suppose we have a stream of length 100, in which eleven different elements appear. The most even distribution of these eleven elements would have one appearing 10 times and the other ten appearing 9 times each. In this case, the surprise number is $10^2 + 10 \times 9^2 = 910$. At the other extreme, one of the eleven elements could appear 90 times and the other ten appear 1 time each. Then, the surprise number would be $90^2 + 10 \times 1^2 = 8110$. □

- Here there is no problem computing moments of any order if we can afford to keep in main memory a count for each element that appears in the stream.
- However, we have seen that if we cannot afford to use that much memory, then we need to estimate the kth moment by keeping a limited number of values in main memory and computing an estimate from these values.
- For the case of distinct elements, each of these values were counts of the longest tail produced by a single hash function. We shall see another form of value that is useful for second and higher moments.

R.M.K GROUP OF INSTITUTIONS

2. The Alon-Matias-Szegedy Algorithm for Second Moments

Let us assume that a stream has a particular length n. Suppose we do not have enough space to count all the mi's for all the elements of the stream. We can still estimate the second moment of the stream using a limited amount of space; the more space we use, the more accurate the estimate will be. We compute some number of variables. For each variable X, we store:

1. A particular element of the universal set, which we refer to as X.element, and

1. An integer X.value, which is the value of the variable. To determine the value of a variable X, we choose a position in the stream between 1 and n, uniformly and at random. Set X.element to be the element found there, and initialize X.value to 1. As we read the stream, add 1 to X.value each time we encounter another occurrence of

Example 4.7: Suppose the stream is $a, b, c, b, d, a, c, d, a, b, d, c, a, a, b$. The length of the stream is $n = 15$. Since a appears 5 times, b appears 4 times, and c and d appear three times each, the second moment for the stream is $5^2 + 4^2 + 3^2 + 3^2 = 59$. Suppose we keep three variables, X_1, X_2, and X_3. Also, assume that at "random" we pick the 3rd, 8th, and 13th positions to define these three variables.

When we reach position 3, we find element c, so we set $X_1.element = c$ and $X_1.value = 1$. Position 4 holds b, so we do not change X_1. Likewise, nothing happens at positions 5 or 6. At position 7, we see c again, so we set $X_1.value = 2$.

At position 8 we find d, and so set $X_2.element = d$ and $X_2.value = 1$. Positions 9 and 10 hold a and b, so they do not affect X_1 or X_2. Position 11 holds d so we set $X_2.value = 2$, and position 12 holds c so we set $X_1.value = 3$. At position 13, we find element a, and so set $X_3.element = a$ and $X_3.value = 1$. Then, at position 14 we see another a and so set $X_3.value = 2$. Position 15, with element b does not affect any of the variables, so we are done, with final values $X_1.value = 3$ and $X_2.value = X_3.value = 2$. □

We can derive an estimate of the second moment from any variable X. This estimate is $n(2X.value - 1)$.

3. Higher-Order Moments

3.1 We estimate kth moments, for k > 2, in essentially the same way as we estimate second moments. The only thing that changes is the way we derive an estimate from a variable.

3.2 We used the formula n(2v – 1) to turn a value v, the count of the number of occurrences of some particular stream element a, into an estimate of the second moment.

3.3 Then, we saw why this formula works: the terms 2v – 1, for v = 1,2,...,m sum to m2, where m is the number of times a appears in the stream.

Notice that $2v-1$ is the difference between v^2 and $(v-1)^2$. Suppose we wanted the third moment rather than the second. Then all we have to do is replace $2v-1$ by $v^3-(v-1)^3 = 3v^2-3v+1$. Then $\sum_{v=1}^{m} 3v^2-3v+1 = m^3$, so we can use as our estimate of the third moment the formula $n(3v^2-3v+1)$, where $v = X.value$ is the value associated with some variable X. More generally, we can estimate kth moments for any $k \geq 2$ by turning value $v = X.value$ into $n\left(v^k-(v-1)^k\right)$.

4. Dealing With Infinite Streams

Technically, the estimate we used for second and higher moments assumes that n, the stream length, is a constant.

4.1 In practice, n grows with time. That fact, by itself, doesn't cause problems, since we store only the values of variables and multiply some function of that value by n when it is time to estimate the moment.

4.2 If we count the number of stream elements seen and store this value, which only requires logn bits, then we have n available whenever we need it.

4.3 A more serious problem is that we must be careful how we select the positions for the variables. If we do this selection once and for all, then as the stream gets longer, we are biased in favor of early positions, and the estimate of the moment will be too large.

R.M.K GROUP OF INSTITUTIONS

4.4 On the other hand, if we wait too long to pick positions, then early in the stream we do not have many variables and so will get an unreliable estimate.

4.5 The proper technique is to maintain as many variables as we can store at all times, and to throw some out as the stream grows.

4.6 The discarded variables are replaced by new ones, in such a way that at all times, the probability of picking any one position for a variable is the same as that of picking any other position.

4.7 Suppose we have space to store s variables. Then the first s positions of the stream are each picked as the position of one of the s variables.

Inductively, suppose we have seen n stream elements, and the probability of any particular position being the position of a variable is uniform, that is s/n.

4.8 When the (n+1)st element arrives, pick that position with probability s/(n+1). If not picked, then the s variables keep their same

4.9 However, if the (n+1)st position is picked, then throw out one of the current s variables, with equal

4.10 Replace the one discarded by a new variable whose element is the one at position n + 1 and whose value is 1.

Surely, the probability that position n + 1 is selected for a variable is what it should be: s/(n + 1).

4.11 However, the probability of every other position also is s/(n + 1), as we can prove by induction on n.

4.12 By the inductive hypothesis, before the arrival of the (n + 1)st stream element, this probability was s/n.

4.13 With probability 1 − s/(n + 1) the (n + 1)st position will not be selected, and the probability of each of the first n positions remains s/n.

4.14 However, with probability s/(n + 1), the (n + 1)st position is picked, and the probability for each of the first n positions is reduced by factor (s−1)/s.

R.M.K GROUP OF INSTITUTIONS

Considering the two cases, the probability of selecting each of the first n positions is

$$(1-\frac{s}{n+1})(\frac{s}{n})+(\frac{s}{n+1})(\frac{s-1}{s})(\frac{s}{n})$$

This expression simplifies to

$$(1-\frac{s}{n+1})(\frac{s}{n})+(\frac{s-1}{n+1})(\frac{s}{n})$$

and then to

$$\left((1-\frac{s}{n+1})+(\frac{s-1}{n+1})\right)(\frac{s}{n})$$

which in turn simplifies to

$$(\frac{n}{n+1})(\frac{s}{n})=\frac{s}{n+1}$$

Thus, we have shown by induction on the stream length n that all positions have equal probability s/n of being chosen as the position of a variable.

11) **Counting Ones in a**

Let us see the counting problems for

Suppose we have a window of length N on a binary stream. We want at all times
.
to be able to answer queries of the form "how many 1's are there in the last k bits?" for any k ≤ N.

As in previous sections, we focus on the situation where we cannot afford to store the entire window. After showing an approximate algorithm for the binary case, we will see how this idea can be extended to summing numbers.

1. The Cost of Exact Counts

To begin, suppose we want to be able to count exactly the number of 1's in the last k bits for any k ≤ N. Then we claim it is necessary to store all N bits of the window, as any representation that used fewer than N bits could not work.

R.M.K GROUP OF INSTITUTIONS

not work. In proof, suppose we have a representation that uses fewer than N bits to represent the N bits in the window. Since there are 2^N sequences of N bits, but fewer than 2^N representations, there must be two different bit strings w and x that have the same representation. Since $w \neq x$, they must differ in at least one bit. Let the last $k-1$ bits of w and x agree, but let them differ on the kth bit from the right end.

Example 4.10: If w = 0101 and x = 1010, then k = 1, since scanning from the right, they first disagree at position 1. If w = 1001 and x = 0101, then k = 3, because they first disagree at the third position from the right.

Suppose the data representing the contents of the window is whatever sequence of bits represents both w and x. Ask the query "how many 1's are in the last k bits?"

1.1 The query-answering algorithm will produce the same answer, whether the window contains w or x, because the algorithm can only see their representation

1.2 But the correct answers are surely different for these two bit-strings. Thus, we have proved that we must use at least N bits to answer queries about the last k bits for any k.

In fact, we need N bits, even if the only query we can ask is "how many 1's are in the entire window of length N?" This is similar to that used above.

1.3 Suppose we use fewer than N bits to represent the window, and therefore we can find w, x, and k as above.

1.4 It might be that w and x have the same number of 1's, as they did in both cases of Example 4.10. However, if we follow the current window by any N – k bits, we will have a situation where the true window contents resulting from w and x are identical except for the leftmost bit, and therefore, their counts of 1's are unequal.

R.M.K GROUP OF INSTITUTIONS

However, since the representations of w and x are the same, the representation of the window must still be the same if we feed the same bit sequence to these representations.
Thus, we can force the answer to the query "how many 1's in the window?" to be incorrect for one of the two possible window contents.

2. The Datar-Gionis-Indyk-Motwani Algorithm

We shall present the simplest case of an algorithm called DGIM. This version of the algorithm uses $O(\log^2 N)$ bits to represent a window of N bits, and allows us to estimate the number of 1's in the window with an error of no more than 50%. Later, we shall discuss an improvement of the method that limits the error to any fraction $\epsilon > 0$, and still uses only $O(\log^2 N)$ bits (although with a constant factor that grows as ϵ shrinks).

To begin, each bit of the stream has a timestamp, the position in which it arrives. The first bit has timestamp 1, the second has timestamp 2, and so on.

2.1 Since we only need to distinguish positions within the window of length N, we shall represent timestamps modulo N, so they can be represented by log2 N bits.

2.2 If we also store the total number of bits ever seen in the stream (i.e., the most recent timestamp) modulo N, then we can determine from a timestamp modulo N where in the current window the bit with that timestamp is.

2.3 We divide the window into buckets, consisting of:

1. The timestamp of its right (most recent) end.
2. The number of 1's in the bucket. This number must be a power of 2, and we refer to the number of 1's as the size of the bucket.

To represent a bucket, we need $\log_2 N$ bits to represent the timestamp (modulo N) of its right end. To represent the number of 1's we only need $\log_2 \log_2 N$ bits. The reason is that we know this number i is a power of 2, say 2^j, so we can represent i by coding j in binary. Since j is at most $\log_2 N$, it requires $\log_2 \log_2 N$ bits. Thus, $O(\log N)$ bits suffice to represent a bucket.

There are six rules that must be followed when representing a stream by buckets.

- The right end of a bucket is always a position with a 1.
- Every position with a 1 is in some bucket.
- No position is in more than one bucket.
- There are one or two buckets of any given size, up to some maximum size.
- All sizes must be a power of 2.
- Buckets cannot decrease in size as we move to the left (back in time)

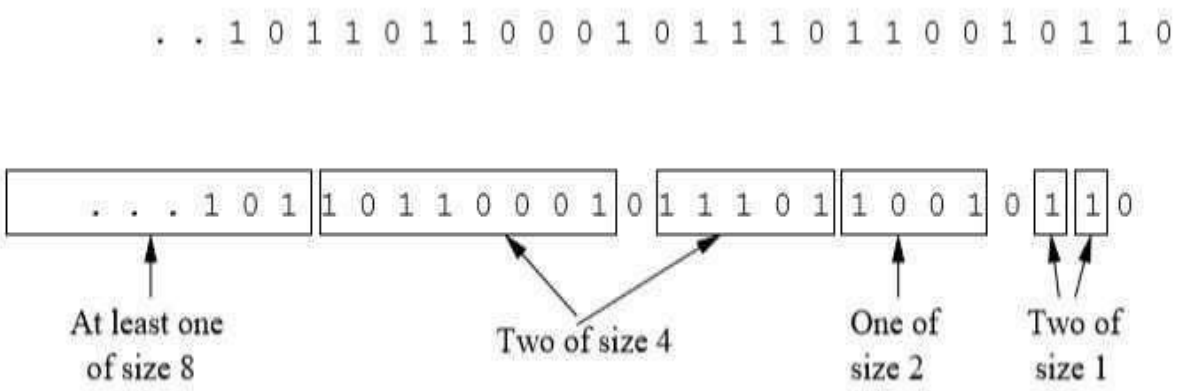


Figure 4.2: A bit-stream divided into buckets following the DGIM rules

Example 4.11: Figure 4.2 shows a bit stream divided into buckets in a way that satisfies the DGIM rules. At the right (most recent) end we see two buckets of size 1.

To its left we see one bucket of size 2. Note that this bucket covers four positions, but only two of them are 1. Proceeding left, we see two buckets of size 4, and we suggest that a bucket of size 8 exists further left.

Notice that it is OK for some 0's to lie between buckets.

Also, observe from Fig. 4.2 that the buckets do not overlap; there are one or two of each size up to the largest size, and sizes only increase moving left.

3. Storage Requirements for the DGIM Algorithm

We observed that each bucket can be represented by $O(\log N)$ bits. If the window has length N, then there are no more than N 1's, surely. Suppose the largest bucket is of size 2^j. Then j cannot exceed $\log_2 N$, or else there are more 1's in this bucket than there are 1's in the entire window. Thus, there are at most two buckets of all sizes from $\log_2 N$ down to 1, and no buckets of larger sizes.

We conclude that there are $O(\log N)$ buckets. Since each bucket can be represented in $O(\log N)$ bits, the total space required for all the buckets representing a window of size N is $O(\log^2 N)$.

4. Query Answering in the DGIM Algorithm

Suppose we are asked how many 1's there are in the last k bits of the window, some $1 \leq k \leq N$. Find the bucket b with the earliest timestamp that includes at least some of the k most recent bits. Estimate the number of 1's to be the sum of the sizes of all the buckets to the right (more recent) than bucket b, plus half the size of b itself.

Example 4.12: Suppose the stream is that of Fig. 4.2, and k = 10. Then the query asks for the number of 1's in the ten rightmost bits, which happen to be 0110010110. Let the current timestamp (time of the rightmost bit) be t. Then the two buckets with one 1, having timestamps t – 1 and t – 2 are completely included in the answer. The bucket of size 2, with timestamp t – 4, is also completely included.

However, the rightmost bucket of size 4, with timestamp t–8 is only partly included.

4.1 We know it is the last bucket to contribute to the answer, because the next bucket to its left has timestamp less than t – 9 and thus is completely out of the window.

4.2 On the other hand, we know the buckets to its right are completely inside the range of the query because of the existence of a bucket to their left with timestamp t – 9 or greater.

4.3 Our estimate of the number of 1's in the last ten positions is thus 6.

4.4 This number is the two buckets of size 1, the bucket of size 2, and half the bucket of size 4 that is partially within range. Of course the correct answer is 5.

Suppose the above estimate of the answer to a query involves a bucket b of size 2^j that is partially within the range of the query. Let us consider how far from the correct answer c our estimate could be. There are two cases: the estimate could be larger or smaller than c.

Case 1: The estimate is less than c. In the worst case, all the 1's of b are actually within the range of the query, so the estimate misses half bucket b, or 2^{j-1} 1's. But in this case, c is at least 2^j; in fact it is at least $2^{j+1} - 1$, since there is at least one bucket of each of the sizes $2^{j-1}, 2^{j-2}, \ldots, 1$. We conclude that our estimate is at least 50% of c.

Case 2: The estimate is greater than c. In the worst case, only the rightmost bit of bucket b is within range, and there is only one bucket of each of the sizes smaller than b. Then $c = 1 + 2^{j-1} + 2^{j-2} + \cdots + 1 = 2^j$ and the estimate we give is $2^{j-1} + 2^{j-1} + 2^{j-2} + \cdots + 1 = 2^j + 2^{j-1} - 1$. We see that the estimate is no more than 50% greater than c.

5. Maintaining the DGIM Conditions

Suppose we have a window of length N properly represented by buckets that satisfy the DGIM conditions. When a new bit comes in, we may need to modify the buckets, so they continue to represent the window and continue to satisfy the DGIM conditions.

First, whenever a new bit enters:

5.1 Check the leftmost (earliest) bucket. If its timestamp has now reached the current timestamp minus N, then this bucket no longer has any of its 1's in the window. Therefore, drop it from the list of buckets.

Now, we must consider whether the new bit is 0 or 1. If it is 0, then no further change to the buckets is needed. If the new bit is a 1, however, we may need to make several changes. First:

5.2 Create a new bucket with the current timestamp and size 1.

If there was only one bucket of size 1, then nothing more needs to be done. However, if there are now three buckets of size 1, that is one too many. We fix this problem by combining the leftmost (earliest) two buckets of size 1.

5.3 To combine any two adjacent buckets of the same size, replace them by one bucket of twice the size. The timestamp of the new bucket is the timestamp of the rightmost (later in time) of the two buckets.

Combining two buckets of size 1 may create a third bucket of size 2. If so, we combine the leftmost two buckets of size 2 into a bucket of size 4. That, in turn, may create a third bucket of size 4, and if so we combine the leftmost two into a bucket of size 8.

This process may ripple through the bucket sizes, but there are at most log2 N different sizes, and the combination of two adjacent buckets of the same size only requires constant time.

As a result, any new bit can be processed in O(logN) time.

R.M.K GROUP OF INSTITUTIONS

Example 4.13: Suppose we start with the buckets of Fig. 4.2 and a 1 enters.

First, the leftmost bucket evidently has not fallen out of the window, so we do not drop any buckets. We create a new bucket of size 1 with the current timestamp, say t. There are now three buckets of size 1, so we combine the leftmost two. They are replaced with a single bucket of size 2. Its timestamp is t – 2, the timestamp of the bucket on the right (i.e., the rightmost bucket that actually appears in Fig. 4.2.

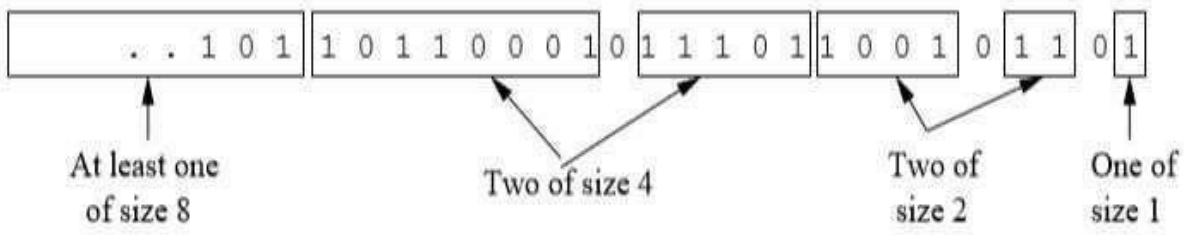


Figure 4.3: Modified buckets after a new 1 arrives in the stream

There are now two buckets of size 2, but that is allowed by the DGIM rules. Thus, the final sequence of buckets after the addition of the 1 is as shown in Fig. 4.3

6. Reducing the Error

Instead of allowing either one or two of each size bucket, suppose we allow either r –1 or r of each of the exponentially growing sizes 1,2,4,..., for some integer r > 2. In order to represent any possible number of 1's, we must relax this condition for the buckets of size 1 and buckets of the largest size present; there may be any number, from 1 to r, of buckets of these sizes.

The rule for combining buckets is essentially the same as in Section 4.6.5. If we get $r + 1$ buckets of size 2^j, combine the leftmost two into a bucket of size 2^{j+1}. That may, in turn, cause there to be $r + 1$ buckets of size 2^{j+1}, and if so we continue combining buckets of larger sizes.

R.M.K GROUP OF INSTITUTIONS

The argument used in Section 6.4 can also be used here.

6.1 However, because there are more buckets of smaller sizes, we can get a stronger bound on the error.

6.2 We saw there that the largest relative error occurs when only one 1 from the leftmost bucket b is within the query range, and we therefore overestimate the true count.

6.3 Suppose bucket b is of size 2j. Then the true count is at least $1 + (r-1)(2^{j-1} + 2^{j-2} + \cdots + 1) = 1 + (r-1)(2^j - 1)$. The overestimate is $2^{j-1} - 1$. Thus, the fractional error is

$$\frac{2^{j-1} - 1}{1 + (r-1)(2^j - 1)}$$

No matter what j is, this fraction is upper bounded by $1/(r-1)$. Thus, by picking r sufficiently large, we can limit the error to any desired $\epsilon > 0$.

7. Extensions to the Counting of Ones

7.1 It is natural to ask whether we can extend the technique of this section to handle aggregations more general than counting 1's in a binary

7.2 An obvious direction to look is to consider streams of integers and ask if we can estimate the sum of the last k integers for any $1 \le k \le N$, where N, as usual, is the window size.

7.3 It is unlikely that we can use the DGIM approach to streams containing both positive and negative integers.

7.4 We could have a stream containing both very large positive integers and very large negative integers, but with a sum in the window that is very close to 0.

7.5 Any imprecision in estimating the values of these large integers would have a huge effect on the estimate of the sum, and so the fractional error could be unbounded.

7.6 For example, suppose we broke the stream into buckets as we have done, but represented the bucket by the sum of the integers therein, rather than the count of 1's.

7.7 If b is the bucket that is partially within the query range, it could be that b has, in its first half, very large negative integers and in its second half, equally large positive integers, with a sum of 0.

7.8 If we estimate the contribution of b by half its sum, that contribution is essentially 0.

7.9 But the actual contribution of that part of bucket b that is in the query range could be anything from 0 to the sum of all the positive integers.

7.10 This difference could be far greater than the actual query answer, and so the estimate would be meaningless.

7.11 On the other hand, some other extensions involving integers do work.

7.12 Suppose that the stream consists of only positive integers in the range 1 to 2m for some m. We can treat each of the m bits of each integer as if it were a separate

7.13 We then use the DGIM method to count the 1's in each bit. Suppose the count of the ith bit (assuming bits count from the low-order end, starting 0) is ci. Then the sum of the integers is

$$\sum_{i=0}^{m-1} c_i 2^i$$

If we use the technique of Section 4.6.6 to estimate each c_i with fractional error at most ϵ, then the estimate of the true sum has error at most ϵ. The worst case occurs when all the c_i's are overestimated or all are underestimated by the same fraction.

R.M.K GROUP OF INSTITUTIONS

12) **Decaying Windows**

- We have assumed that a sliding window held a certain tail of the stream, either the most recent N elements for fixed N, or all the elements that arrived after some time in the past.
- Sometimes we do not want to make a sharp distinction between recent elements and those in the distant past, but want to weight the recent elements more heavily.
- In this section, we consider "exponentially decaying windows," and an application where they are quite useful: finding the most common "recent" elements.

1. The Problem of Most-Common Elements

1.1 Suppose we have a stream whose elements are the movie tickets purchased all over the world, with the name of the movie as part of the element

1.2 We want to keep a summary of the stream that is the most popular movies "currently." While the notion of "currently" is imprecise, intuitively, we to discount the popularity of a movie like Star Wars–Episode 4, which sold many tickets, but most of these were sold decades

1.3 [a]O[g]n[o]t·he other hand, a movie that sold n tickets in each of the last 10 weeks is probably more popular than a movie that sold 2n tickets last week but nothing in previous weeks.

1.4 One solution would be to imagine a bit stream for each movie. The ith bit has value 1 if the ith ticket is for that movie, and 0 otherwise.

1.5 Pick a window size N, which is the number of most recent tickets that would be considered in evaluating popularity.

1.6 Then, use the method of Section 6 to estimate the number of tickets for each movie, and rank movies by their estimated counts.

1.7 This technique might work for movies, because there are only thousands movies, but it would fail if we were instead recording the popularity of items sold at Amazon, or the rate at which different Twitter-users tweet, because there are too many Amazon products and too many tweeters. Further, it only offers approximate answers.

2. Definition of the Decaying Window

An alternative approach is to redefine the question so that we are not asking for a count of 1's in a window. Rather, let us compute a smooth aggregation of all the 1's ever seen in the stream, with decaying weights, so the further back in the stream, the less weight is given. Formally, let a stream currently consist of the elements $a_1, a_2, \ldots, a_t$, where a_1 is the first element to arrive and a_t is the current element. Let c be a small constant, such as 10^{-6} or 10^{-9}. Define the *exponentially decaying window* for this stream to be the sum

$$\sum_{i=0}^{t-1} a_{t-i}(1-c)^i$$

The effect of this definition is to spread out the weights of the stream elements as far back in time as the stream goes. In contrast, a fixed window with the same sum of the weights, $1/c$, would put equal weight 1 on each of the most recent $1/c$ elements to arrive and weight 0 on all previous elements. The distinction is suggested by Fig. 4.4.

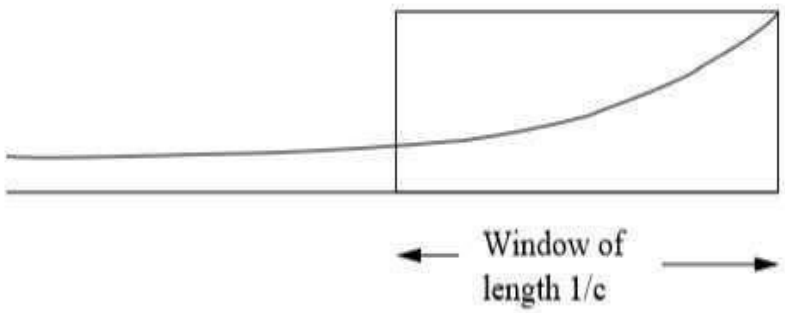


Figure 4.4: A decaying window and a fixed-length window of equal weight

R.M.K GROUP OF INSTITUTIONS

It is much easier to adjust the sum in an exponentially decaying window than in a sliding window of fixed length.

In the sliding window, we have to worry about the element that falls out of the window each time a new element arrives. That forces us to keep the exact elements along with the sum, or to use an approximation scheme such as DGIM. However, when a new element at+1 arrives at the stream input, all we need to do is:

1. Multiply the current sum by $1 - c$.
2. Add a_{t+1}.

The reason this method works is that each of the previous elements has now moved one position further from the current element, so its weight is multiplied by $1 - c$. Further, the weight on the current element is $(1 - c)^0 = 1$, so adding a_{t+1} is the correct way to include the new element's contribution.

3. Finding the Most Popular

E

- Let us return to the problem of finding the most popular movies in a stream of ticket

3.1 We shall use an exponentially decaying window with a constant c, which .we might think of as 10^9. That is, we approximate a sliding window holding the last one billion ticket sales.

3.2 For each movie, we imagine a separate stream with a 1 each time a ticket for that movie appears in the stream, and a 0 each time a ticket for some other movie arrives.

3.3 The decaying sum of the 1's measures the current popularity of the movie.

3.4 We imagine that the number of possible movies in the stream is huge, so we do not want to record values for the unpopular movies.

R.M.K GROUP OF INSTITUTIONS

Therefore, we establish a threshold, say 1/2, so that if the popularity score for a movie goes below this number, its score is dropped from the counting.

For reasons that will become obvious, the threshold must be less than 1, although it can be any number less than 1. When a new ticket arrives on the stream, do the following:

1.For each movie whose score we are currently maintaining, multiply its score by (1 – c).

1.Suppose the new ticket is for movie M. If there is currently a score for M, add 1 to that score. If there is no score for M, create one and initialize it to 1.

3. If any score is below the threshold 1/2, drop that score.

3.5 It may not be obvious that the number of movies whose scores are maintained at any time is

3.6 However, note that the sum of all scores is

3.7 There cannot be more than 2/c movies with score of 1/2 or more, or else the sum of the scores would exceed 1/c.

3.8 Thus, 2/c is a limit on the number of movies being counted at any time.

3.9 Of course in practice, the ticket sales would be concentrated on only a small number of movies at any time, so the number of actively counted movies would be much less than 2/c.

R.M.K GROUP OF INSTITUTIONS

VIDEO LINKS

Unit – IV

VIDEO LINKS

Sl. No.	Topic	Video Link
1	The data stream model	https://www.youtube.com/watch?v=nbBJ27XhEyM
2	sampling data in a stream	https://www.youtube.com/watch?v=895jWrdNA5I&t=11 9s
3	filtering streams	https://www.youtube.com/watch?v=qBTdukbzc78
4	Counting Ones in Window	https://www.youtube.com/watch?v=NDSAiEGJshM
5	Decaying Windows	https://www.youtube.com/watch?v=SRTSVxUnsNI

R.M.K GROUP OF INSTITUTIONS

10. ASSIGNMENT : UNIT – IV

1. Differentiate between filtering streams and analyzing streams with examples.
2. Apply a sampling technique on a continuous stream of sensor readings and explain how representative samples are obtained.
3. Given two streaming datasets (Twitter and Stock Market), analyze which filtering strategy would be more effective in each case and justify.
4. Develop a model for real-time sentiment analysis using the counting ones in a window approach.
5. Propose a streaming solution that balances accuracy and memory usage when monitoring website click streams.

R.M.K GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

PART-AQ&A UNIT-IV

R.M.K
GROUP OF
INSTITUTIONS

11. PART A : Q & A : UNIT – IV

1. What is an archival store? (CO4, K4)

- Streams may be archived in a large archival store, but we assume it is not possible to answer queries from the archival store.
- It could be examined only under special circumstances using time-consuming retrieval processes.

2. What is the purpose of working store in data stream management system? (CO4, K4)

- In a working store, Summaries or parts of streams may be placed, and it can be used for answering queries.
- The working store might be disk, or it might be main memory, depending on how fast we need to process queries.
- But either way, it is of sufficiently limited capacity that it cannot store all the data from all the streams.

3.What are the examples of Stream Sources? (CO4, K4)

- Sensor Data
- Image Data
- Internet and Web Traffic

4. What are the Stream queries ? (CO4, K4)

- There are two ways that queries get asked about streams. In data stream management system, there is a place within the processor where standing queries are stored.
- These queries are permanently executing and produce outputs at appropriate times.
- Example Queries : the maximum temperature ever recorded by that sensor, Web sites often like to report the number of unique users over the

11. PART A : Q & A : UNIT – IV

past month. If we think of each login as a stream element, we can maintain a window that is all logins in the most recent month.

5. What do you mean by ad-hoc Query ? (CO4, K4)

- The other form of query is ad-hoc, a question asked once about the current state of a stream or streams. If we do not store all streams in their entirety, as normally we can not, then we cannot expect to answer arbitrary queries about streams.
- If we have some idea what kind of queries will be asked through the ad-hoc query interface, then we can prepare for them by storing appropriate parts or summaries of streams.
- If we want the facility to ask a wide variety of ad-hoc queries, a common approach is to store a sliding window of each stream in the working store.

6. What is a sliding window ? (CO4, K4.)

A sliding window can be the most recent n elements of a stream, for some n, or it can be all the elements that arrived within the last t time units, e.g., one

- If we regard each stream element as a tuple, we can treat the window as a relation and query it with any SQL query.
- The stream-management system must keep the window fresh, deleting the oldest elements as new ones come in.

7. How hash function is used in sampling data stream ? (CO4, K4)

- By using a hash function, one can avoid keeping the list of users.That is, we hash each user name to one of ten buckets, 0 through 9. If the user hashes to bucket 0, then accept this search query for the sample, and if not, then not.

R.M.K GROUP OF INSTITUTIONS

11. PART A : Q & A : UNIT – IV

- Effectively, we use the hash function as a random-number generator, with the important property that, when applied to the same user several times, we always get the same "random" number.
- That is, without storing the in/out decision for any user, we can reconstruct that decision any time a search query by that user arrives.

8. What is general sampling problem ? (CO4, K4)
   - Our stream consists of tuples with n components. A subset of the components are the key components, on which the selection of the sample will be based.
   - There are three components as user, query, and time , of which only user is in the key.
   - However, we could also take a sample of queries by making query be the key, or even take a sample of user-query pairs by making both those components form the
   - To take a sample of size a/b, we hash the key value for each tuple to b buckets, and accept the tuple for the sample if the hash value is less than a.

9. What do you mean by filtering of streams? (CO4, K4)
   - Another common process on streams is selection, or filtering.
   - We want to accept those tuples in the stream that meet a criterion. Accepted tuples are passed to another process as a stream, while other tuples are dropped.
   - If the selection criterion is a property of the tuple that can be calculated (e.g., the first component is less than 10), then the selection is easy to do.
   - The problem becomes harder when the criterion involves lookup for membership in a set.

R.M.K GROUP OF INSTITUTIONS

11. PART A : Q & A : UNIT – IV

10. What is Bloom Filtering ? (CO4, K4)

- The technique known as "Bloom filtering" has a way to eliminate most of the tuples that do not meet the criterion.
- A Bloom filter consists of:
  1. An array of n bits, initially all 0's.
  2. A collection of hash functions h1,h2,...,hk.
     Each hash function maps "key" values to n buckets, corresponding to the n bits of the bit-array.
  3. A set S of m key values.
- The purpose of the Bloom filter is to allow through all stream elements whose keys are in S, while rejecting most of the stream elements whose keys are not in S.

11. What is the Count-Distinct Problem ? (CO4, K2)

- Suppose stream elements are chosen from some universal
- We would like to know how many different elements have appeared in the stream, counting either from the beginning of the stream or from some known time in the
- As a useful example of this problem, consider a Web site gathering statistics on how many unique users it has seen in each given month.

12. What is an universal set in Count-Distinct Problem ? Give example. (CO4, K4)

- The universal set is the set of logins for that site, and a stream element is generated each time someone logs in. This measure is appropriate for a site like Amazon, where the typical user logs in with their unique login name.
- A similar problem is a Web site like Google that does not require login to issue a search query, and may be able to identify users only by the IP address from which they send the query.

11. PART A : Q & A : UNIT – IV

- There are about 4 billion IP addresses, 2 sequences of four 8-bit bytes will serve as the universal set in this case.

13. What is Flajolet-Martin Algorithm ? (CO4, K4)

- The important property of a hash function is that when applied to the same element, it always produces the same result. This property was also essential for the sampling technique.
- The idea behind the Flajolet-Martin Algorithm is that the more different elements we see in the stream, the more different hash-values we shall see.
- As we see more different hash-values, it becomes more likely that one of these values will be "unusual." The particular unusual property we shall exploit is that the value ends in many 0's, although many other options exist.

14. What is Estimating Moments ? (CO4, K4)

Here we consider a generalization of the problem of counting distinct elements in a

- The problem, called computing "moments," involves the distribution of frequencies of different elements in the stream.
- We shall define moments of all orders and concentrate on computing second moments, from which the general algorithm for all moments is a simple extension.

15. What is the Alon-Matias-Szegedy Algorithm for Second Moments ? (CO4, K4)

- Let us assume that a stream has a particular length n.
- Suppose we do not have enough space to count all the mi's for all the elements of the stream. We can still estimate the second moment of the

11. PART A : Q & A : UNIT – IV

stream using a limited amount of space; the more space we use, the more accurate the estimate will be. We compute some number of variables.

16. How do you deal with infinite streams ? (CO4, K4)

- The proper technique is to maintain as many variables as we can store at all times, and to throw some out as the stream grows.
- The discarded variables are replaced by new ones, in such a way that at all times, the probability of picking any one position for a variable is the same as that of picking any other position.
- Suppose we have space to store s variables. Then the first s positions of the stream are each picked as the position of one of the s variables.

17. What are the six rules that must be followed when representing a stream by buckets? (CO4, K4)

- The right end of a bucket is always a position with a 1.
- Every position with a 1 is in some bucket.
- No position is in more than one bucket.
- There are one or two buckets of any given size, up to some maximum size.
- All sizes must be a power of 2.
- Buckets cannot decrease in size as we move to the left (back in time).

18. What is Query Answering in the DGIM Algorithm ? (CO4, K4)

- Suppose we are asked how many 1's there are in the last k bits of the window, for some $1 \leq k \leq N$.
- Find the bucket b with the earliest timestamp that includes at least some of the k most recent bits.
- Estimate the number of 1's to be the sum of the sizes of all the buckets to the right (more recent) than bucket b, plus half the size of b itself.

R.M.K GROUP OF INSTITUTIONS

11. PART A : Q & A : UNIT – IV

19. What is Decaying Windows ? (CO4, K4)

- We have assumed that a sliding window held a certain tail of the stream, either the most recent N elements for fixed N, or all the elements that arrived after some time in the past.
- Sometimes we do not want to make a sharp distinction between recent elements and those in the distant past, but want to weight the recent elements more heavily.

R.M.K GROUP OF INSTITUTIONS

12. PART B QUESTIONS : UNIT – IV

1. Explain the Stream Data Model in detail. (CO4, K4)
2. Write short notes on Examples of Stream Sources. (CO4, K4)
3. Explain Stream Queries with examples. (CO4, K4)
4. What is Sampling Data in a Stream? Explain the General Sampling Problem. (CO4, K4)
5. What is Filtering Streams? Explain the Analysis of Bloom Filtering. (CO4, K4)
6. Explain the concept of Counting Distinct Elements in a Stream. (CO4, K4)
7. Write short notes on Estimating Moments. How do you deal with Infinite Streams. (CO4, K4)
8. Explain the concept of Counting Ones in Window in detail. (CO4, K4)
9. Write short notes on Decaying Windows. (CO4, K4)

R.M.K GROUP OF INSTITUTIONS

13. SUPPORTIVE ONLINE CERTIFICATION COURSES

NPTEL : https://onlinecourses.nptel.ac.in/noc21_cs45/preview

coursera : https://www.coursera.org/professional-certificates/google-data-analytics

Udemy : https://www.udemy.com/topic/data-analysis/

Mooc : https://www.mooc-list.com/tags/data-analytics

edx : https://www.edx.org/learn/data-analytics

R.M.K GROUP OF INSTITUTIONS

14. REAL TIME APPLICATIONS

Real-time data streaming and analytics is the process used for analyzing the huge amount of data at the moment it is used or produced.

In this, we extract valuable information for the organization as soon as it's created or stored.

Let's take an example of analyzing the huge amount of data as it is produced within banks and branches, stock exchanges throughout the globe and further, this analytics data delivers to the users/administrator through an analytics dashboard. Real-time analytics can be used for the below purposes:-

- To report historical data and current data concurrently.
- For receiving alerts based on certain and predefined parameters
- To build operational decisions and apply them to business processes or on other production activities based on a real-time and ongoing basis
- To apply pre-existing prescriptive models or predictive models
- For the outlook of real-time displays or dashboards in real-time on constantly changing datasets

Data Visualization

A set of historical datasets can be placed into a single screen to represent an overall point. Still, on the other hand, streaming data can be visualized so that it updates in real-time to display what is occurring every second.

Business Insights

When it's about business, real-time analytics can be used for receiving alerts based on certain and predefined parameters. For example, if any store there is a drop in sales, then an alert can be triggered to tell management about the serious problem. Increase competitiveness: Real-time analytics helps companies surpass competitors who are still based on batch processing analysis.

Security

Take an example of fraud detection, fraud can be detected immediately whenever it happens, and a proper safety precaution can be taken to limit the damage.

R.M.K GROUP OF INSTITUTIONS

15. CONTENTS BEYOND SYLLABUS : UNIT – IV

Apache NIFI

In simple words, we can say that Apache NiFi is a data ingestion tool. It is an integrated data logistics platform for automating the movement of data between disparate systems.

It provides real-time control that makes it easy to manage data movement between any source and any destination. Apache NiFi supports disparate and distributed sources of differing formats, schemas, protocols, speeds, and sizes such as machines, geolocation devices, clickstreams, files, social feeds, log files and videos, and more.

It is configurable plumbing for moving data around, similar to how FedEx, UPS, or other courier delivery services move parcels around. Apache NiFi also allows us to trace our data in real-time, just like we could trace a delivery.

Apache StreamSets

StreamSets is also a data ingestion tool similar to NIFI. Basically, StreamSets is a data operations platform where we can efficiently develop batch and streaming dataflows, and further operate them with full visibility and control, and easily evolve our architecture over time.

Apache Beam is an open-source unified programming model used to define and execute data processing pipelines.

Real-Time Stream Storage Sources

Further, we need storage in which we can ingest the stream. Many open-source stream storages are available in the market. Some of them are below:-

Apache Kafka

Kafka is beneficial for building real-time data pipelines and streaming apps. It is horizontally scalable, fault-tolerant, wicked fast, and runs in production in thousands of companies.

Apache Pulsar

Apache Pulsar is an open-source distributed pub-sub messaging system created at Yahoo and now part of the Apache Software Foundation.

NATS.IO

NATS Server is a simple, high-performance open-source messaging system for cloud-native applications, IoT messaging, and microservices architectures.

R.M.K GROUP OF INSTITUTIONS

Assessment Schedule (Proposed Date & Actual Date)

R.M.K GROUP OF INSTITUTIONS

16. Assessment Schedule (Proposed Date & Actual Date)

Sl. No.	ASSESSMENT	Proposed Date	Actual Date
1	FIRST INTERNAL ASSESSMENT		
2	SECOND INTERNAL ASSESSMENT		
3	MODEL EXAMINATION		
4	END SEMESTER EXAMINATION		

R.M.K GROUP OF INSTITUTIONS

17. PRESCRIBED TEXT BOOKS & REFERENCE BOOKS

TEXT BOOKS:

1. Subhashini Chellappan, Seema Acharya, "Big Data and Analytics", 2nd edition, Wiley Publications, 2019.
2. Suresh Kumar Mukhiya and Usman Ahmed, "Hands-on Exploratory Data Analysis with Python", Packt publishing, March 2020.
3. Jure Leskovek, Anand Rajaraman and Jefrey Ullman," Mining of Massive Datasets. v2.1", Cambridge University Press,2019.
4. Glenn J. Myatt, Wayne P. Johnson, Making Sense of Data II : A Practical Guide To Data Visualization, Advanced Data Mining Methods, and Applications, Wiley 2009.

REFERENCES:

1. Nelli, F., Python Data Analytics: with Pandas, NumPy and Matplotlib, Apress, 2018.
2. Bart Baesens," Analytics in a Big Data World: The Essential Guide to DataScience and its Applications", John Wiley & Sons, 2014
3. Min Chen, Shiwen Mao, Yin Zhang, Victor CM Leung, Big Data: Related Technologies, Challenges and Future Prospects, Springer, 2014.
4. Michael Minelli, Michele Chambers, Ambiga Dhiraj, "Big Data, Big Analytics: Emerging Business Intelligence and Analytic Trends", John Wiley & Sons, 2013.
5. Marcello Trovati, Richard Hill, Ashiq Anjum, Shao Ying Zhu, "Big Data Analytics and cloud computing – Theory, Algorithms and Applications", Springer International Publishing, 2016

R.M.K GROUP OF INSTITUTIONS

18. MINI PROJECT SUGGESTION

Mini Project:

- Implement a big data processing pipeline using Hadoop and Spark for large datasets.
- Analyze supply chain data to optimize inventory levels and reduce costs.
- Use data analytics techniques to detect unusual patterns in network traffic that may indicate security threats.
- Analyze traffic data to predict traffic flow and congestion in urban areas.
- Build a recommendation system for an e-commerce platform using collaborative filtering or content-based filtering.
- The project should contain the following components
- Realtime dataset
- Data preparation & Transformation
- Handling missing Data
- Data Storage
- Algorithm for data analytics
- Data visualization: Charts, Heatmap, Crosstab, Treemap

R.M.K GROUP OF INSTITUTIONS

Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.